

POINTERS

What is a pointer ?

- A Pointer is a derived Data Type.
- A pointer is nothing but a variable that contains the address which is a location of the another variable in memory.

Benefits of using the pointer :

- Pointer is more efficient in handling array and data tables.
- Pointer reduces the length and complexity of the program.
- The use of pointer array to character string results in saving of data storage space in memory.
- Pointer can be used to return multiple values from functions via functions arguments.
- Pointers permit references to functions.
- Pointers allow C to support dynamic memory management.
- Pointers provide an efficient tool for manipulating Dynamic Data Structure such as Linked List, Queue, Stack and Tree.
- They increase the execution speed.

Title	Syntax	Example
1. 1. Declaring a Pointer	<code>data_type *pointer_name;</code>	<code>int *p; // pointer to int</code> <code>float *fp; // pointer to float</code> <code>char *cp; // pointer to char</code> • p will store the address of an integer • fp will store the address of a float • cp will store the address of a char
2. Initializing a Pointer	You must store the address of a variable into the pointer using the & (address-of) operator.	<code>int a = 10;</code> <code>int *p = &a;</code> • a stores 10 • &a is the address of a • p stores that address
3. Accessing Value Using Pointer (Dereferencing)	Use *pointer_name to access the value stored at the address.	<code>printf("%d", *p);</code> // prints 10
4. Null Pointer Initialization	A pointer can also be initialized to a NULL address.	<code>int *p = NULL;</code> This means the pointer points to nothing.
5. Pointer Initialization After Declaration		<code>int a = 5;</code> <code>int *p; // declaration</code> <code>p = &a; // initialization</code>

✓ Full Working Example

```
#include <stdio.h>
int main() {
    int a = 10;           // normal variable
    int *p = &a;          // pointer initialized with address of a

    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    printf("Pointer p stores = %p\n", p);
    printf("Value at pointer p = %d\n", *p);
    return 0;
}
```

OUTPUT:-

```
Value of a = 10
Address of a = 0x7ffebc9db044
Pointer p stores = 0x7ffebc9db044
Value at pointer p = 10
```

Pointers expressions:

In C programming, a **pointer expression** is any valid combination of pointers, variables, and operators that evaluates to an address or a value stored at an address.

Core Pointer Operators

- **Address-of Operator (&):** Returns the memory address of a variable.
- **Dereferencing Operator (*):** Accesses the value stored at the address held by the pointer.

Types of Pointer Expressions

A. Assignment Expressions

This is the simplest form, where you assign an address to a pointer variable.

```
int x = 10;
int *p;
p = &x; // 'p' now holds the address of 'x'
```

B. Pointer Arithmetic Expressions

You can add or subtract integers from pointers. However, the movement isn't in bytes; it's in **data type units**.

- `p + 1`: Points to the next memory location of the same data type.
- `p - 1`: Points to the previous memory location.

Note: If `p` is an integer pointer (4 bytes) and starts at address 1000, `p + 1` will result in 1004.

C. Pointer Comparison Expressions

You can compare two pointers using relational operators (`==`, `!=`, `<`, `>`). This is mostly used when traversing arrays to check if you've reached the end.

```
if (ptr1 == ptr2) {
    // Both pointers point to the same memory location
}
```

Shorthand Pointer Expressions

Expression	Meaning
<code>*p++</code>	Access value at <code>p</code> , then increment the pointer address.
<code>*++p</code>	Increment the pointer address first, then access the value.
<code>(*p)++</code>	Access the value at <code>p</code> , then increment the value itself.

Pointer and Arrays

- When an array is declared, the compiler allocates a base address and sufficient amount of storage to contain all the elements of the array in memory location
- Array name stores the **base address**
- Base address = address of first element

```
int x[5] = {1,2,3,4,5};
```

Element	<code>x[0]</code>	<code>x[1]</code>	<code>x[2]</code>	<code>x[3]</code>	<code>x[4]</code>
Value	1	2	3	4	5
Address	1000	1002	1004	1006	1008

(Assuming each integer = 2 bytes)

Pointers and Character Strings

- String = array of characters ending with '\0'

<pre>char str[] = "Hello"; char *p = str;</pre>	<pre>→ p points to 'H' printf("%c", *p); // H printf("%c", *(p+1)); // e</pre>
---	--

Pointers and Structures

✓ Accessing structure members using pointer

```
struct Student {  
    int roll;  
    char name[20];  
};  
  
struct Student s = {1, "Riya"};  
struct Student *p = &s;  
  
printf("%d", p->roll);      // same as (*p).roll  
printf("%s", p->name);
```

Pointers and Functions

Call By Value

It is a method used to call a function when we do not want any changes in the actual parameters. In the call by value method, the value of actual parameters are copied to the formal parameters, i.e., the different memory areas are allocated for actual parameters which are used in the function call and formal parameters which are used in the function definition, as a result, any changes made in the formal parameters don't affect the actual parameters.

Example

```
#include<stdio.h>  
void Increment(int x){  
    x+=1;  
    printf("Value of X from the function : %d",x);  
}  
int main(){  
    int x=10;  
    Increment(x);  
    printf("\nValue of X from the main : %d",x);  
}
```

Output:-

Value of X from the function : 11

Value of X from the main : 10

Call By Reference

It is a method used to call a function when any changes made to the formal parameters are to be reflected in the actual parameters. In the call by reference method, the address of the actual parameters is passed to the function call as the formal parameters, i.e., memory allocation for both the actual and formal parameters is the same. Therefore any changes made to formal parameters are actually being made to the actual parameters only, and hence their values are in sync.

Example

```
#include<stdio.h>
```

```
void Increment(int *x){  
    *x+=1;  
    printf("Value of X from the function : %d",*x);  
}
```

```
int main(){  
    int x=10;  
    Increment(&x);  
    printf("\nValue of X from the main : %d",x);  
}
```

Output

Value of X from the function : 11

Value of X from the main : 11